

for each object constructor
execute only once

To initialize instance variable
we can use constructor

when we create an object
of class then constructor
will execute automatically

Constructor not having any
return type including void.

for each object method
execute no. of times.

To write business logic
we can use method.

we have to call for
method execution

methods having return type.

* Abstraction *

* The process of hiding the implementation details from the user, only the essential functionality will be provided to the user.

* The user will have the information on what the object does instead of how it does.

→ Example of ATM.

for each object constructor
execute only once

To initialize instance variable
we can use constructor

When we create an object
of class then constructor
will execute automatically

Constructor not having any
return type including void.

for each object method
execute no. of times.

To write business logic
we can use method.

we have to call for
method execution

methods having return type

* Abstraction *

* The process of hiding the implementation details from
the user, only the essential functionality will be provided
to the user.

* The user will have the information on what the object
does instead of how it does.

→ Example of ATM.

we can see the diff. functionality options (withdraw,
check balance, change PIN etc) provided on ATM screen
but can't see the actual program developed to run
these functionalities.

* Ways to Achieve Abstraction:

* There are two ways.

① Abstract class

② Interface

* Abstract class

→ class which contains the abstract keyword in its declaration it is known as abstract class.

syntax

```
abstract class className {
```

* Imp :- ① By using abstract classes we can achieve partial abstraction.

② we can achieve 0-100% abstraction using abstract class

③ abstract classes may have or may not contain abstract method.

Q. what is abstract methods?

→ method without body called as abstract method.

syntax, abstract return type methodName();

④ if the class is having atleast one abstract method then compulsory we have to define the class as abstract class.

⑤ if you inherit abstract class we have to provide implementations to all the abstract method in child class.

* Note, we can't create object of abstract class.

ex. ⑦ abstract class Test {

abstract void add();

}

Calculation extends Test

```
void add() {  
    int a = 10, b = 20, c;  
    c = a + b;  
    sysout(c);  
}
```

```
class main {  
    p. s. v. m. {
```

```
    Test t1 = new Calculation();  
    t1.add();  
}
```

O/P = 30

Interface

- * Like a class, an interface is also reference datatype.
- * Like a class, an interface is used to represent real world entities.
- * Interfaces is a fully unimplemented class.
- * Interfaces are similar to abstract class but having all the methods of abstract type.
- * interfaces are the blueprint of the class. It specify what a class must do & not how.

IMP * Interfaces are used to achieve abstraction.

- * By using interface we can achieve 100% abstraction.
- * By using interface we can achieve full abstraction in java.

- * we can not create object of interface
- * all methods in an interface is abstract.

Syntax:
 interface InterfaceName {
 public static final Variables
 public abstract Methods;
 }

Notes:- ① Interface does not contains any constructor.

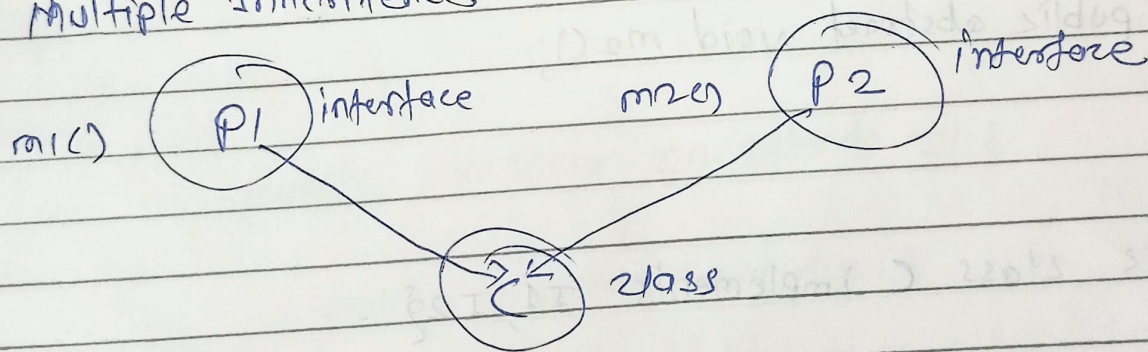
② Interface can not contains instance variables.

③ By default interface can contains public abstract method.
 public static final variable.

④ —||—

⑤ we have to implements all abstract methods in child classes.

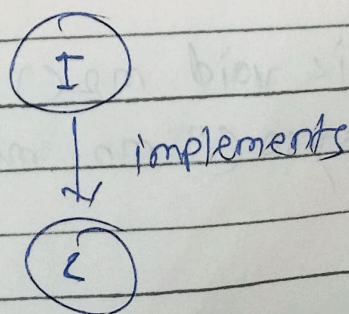
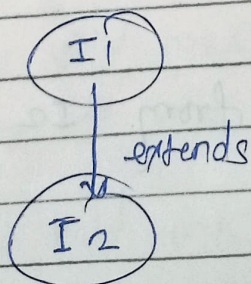
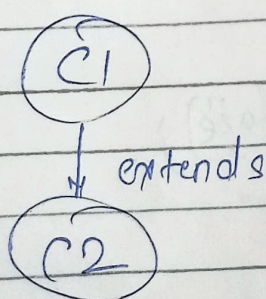
* Multiple Inheritance



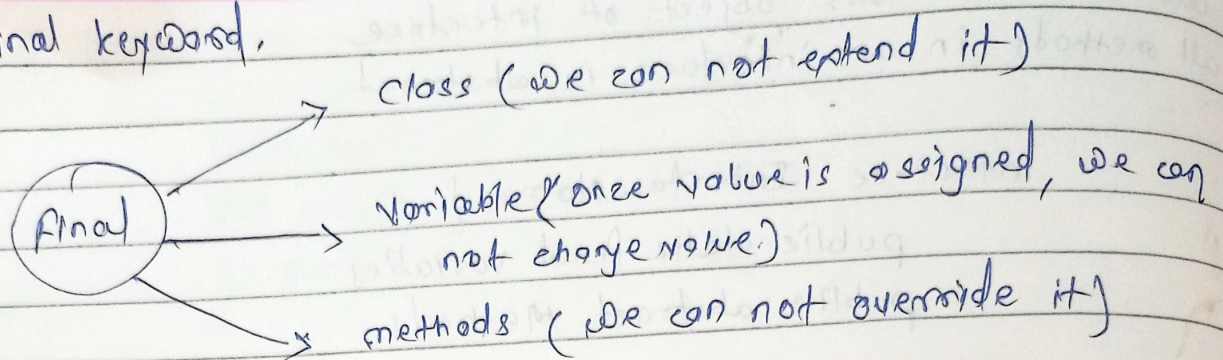
C c = new C ();

c. m1 ();

c. m2 ();



* final keyword,



ex. public interface I1 {

public abstract void m1();

}

public interface I2 {

public abstract void m2();

}

public class C implements I1, I2 {

public void m1() {

System.out.println("m1 method from I1 interface");

}

public void m2() {

System.out.println("m2 method from I2 interface");

}

public class Main {

p.s.v.m.g

C c = new C();

c.m1();

c.m2();

OP $\Rightarrow$ m1 method from I1 interface
m2 method from I2 interface

ex 2. public interface I1 {

public abstract void m1();

}

public interface I2 extends I1 {

public abstract void m2();

}

public class C implements I2 {

public void m2() {

sysout("m2 method from I2 interface");

}

public void m1() {

sysout("m1 method from I1 method");

}

public class Main {

p.s.v.m {
C c = new C();
c.m1();
c.m2();

}

2/p2) m1 method from I1 interface
—IL

ex. 3. public class P {

final void m1() {
sysout("P class from m1 method");
}

public class Z extends P {

final int x = 888;
void m1() {
sysout("Z class from m1 method");
}

P.s.v.m {

C c = new C();

c.m1();

Array

* Need!.. To represent group of variable values / elements as a single entity.

* define array!..

→ Array is a collection of similar data type of elements.

* Advantage!

→ code optimization!

- we can sort & retrieve elements efficiently.

→ random access

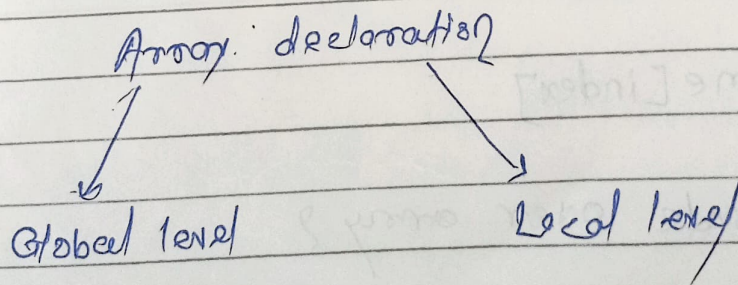
- array support for indexing. we can get random data using index.

* Disadvantage:

→ Fixed in size.

* Imp points!

- ① Array can store similar datatype elements only.
- ② Array is fixed in size.
- ③ Array used to base indexing.
- ④ Array elements stored in contiguous memory locations.
- ⑤ Using indexing we can access random element from array.
- ⑥ array is a referenced datatype.



Class ClassName {

datatype[] varname;

}

Class classname {

~ test() {

datatype[] varname;

}

}

* Array declaration:

```
datatype[] varname;
```

* Array object creation:

```
varname = new datatype[size];
```

* Array variable initialization:

```
varname[index] = value;
```

* Declaration, initialization of array:

```
datatype[] varname = { value, value, value };
```

Q. How to store data in array?

sty:

```
datatype[] varname = new datatype[size];
```

```
varname[index] = value;
```

Q. How to access data from array:

sty:

```
varname[index]
```

Q. How to iterate over array?

⇒ using for loop & for each loop.

* foreach loop !

for (datatype varname : sequence) {

 // body
}

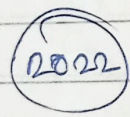
* → for each element present in sequence foreach loop body will execute,

→ taking out one by one element from a sequence & for each element execute body.

Ex.

```
int[] number = new int[5];  
number[0] = 10;  
number[1] = 20;  
number[2] = 30;  
number[3] = 40;  
number[4] = 50;
```

* memory representation of array!

	2022	2025	2030	2037	2048
	10	20	30	40	50
number	0	1	2	3	4

ex.

```
public class Demo {
```

```
    String[] names;
```

```
    String[] getNames() {
```

```
        names = new String[5];
```

```
names[0] = "Jag";  
names[1] = "Kiran";  
names[2] = "Paran";  
names[3] = "Ajay";  
names[4] = "Akshay";
```

```
return names;
```

```
}
```

```
p.s. v.m. §
```

```
demo dd = new demo();  
String[] arr = dd.getNames();
```

```
for (String name : arr) §  
    syso(name);  
}
```

String

- * String is a non-primitive datatype.
- * String is a sequence of characters.
- * String is an array of characters.
- * ~~Java~~ java String is basically an object that represents sequences of characters.
- * An array of characters work same as String.
- * String class implements CharSequence, Serializable, Comparable interface.
- * String class is final so we can extend String class.

```
public final String extends Object implements CharSequence,  
Serializable, Comparable {
```

```
-----  
// methods  
-----
```

```
}
```

→ In java strings class is immutable

→ There are four ways to create string in java

- Using String literal
- Using new keyword
- Using StringBuffer class
- Using StringBuilder class

Note: ① characters are enclosed in ' ' ;
② string are enclosed in " " ;

Q. How to create String in Java 21

- Using string literal:

```
String varname = "characters";
```

③ Using new keyword!

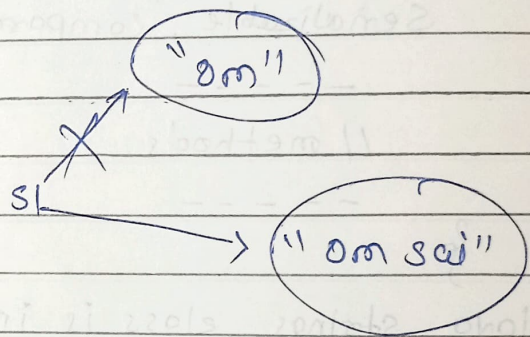
String name = new String ("string literal")

String Is Immutable & ?

→ Immutable means non-changeable.

→ once we create an object there is no way to change in that object if we are trying to change in that object by default new object got created with those changes called as immutable objects.

Ex. String s1 = "om";
String s1 = "om sai";



* == operator in strings!

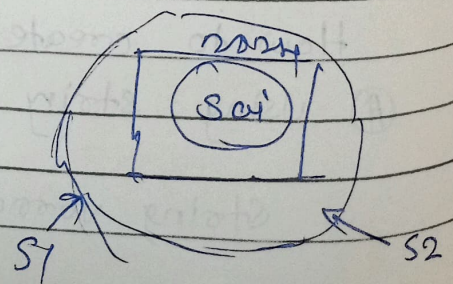
→ If both reference variables are pointing to some memory location / some object then == operator will return true value, else it will return false.

→ It is used to compare memory address of reference variable

Ex. String s1 = "sai";
String s2 = "sai";

System.out.println(s1 == s2); // true

SEP AREA



equals() method on string:
→ It is used to compare content of two strings.
→ If the content of two strings are same then it will return true, else it will return false.
→ equals method will compare characters of two given strings.

* Ex. String s1 = "sal";
String s2 = "sal";
String s3 = "som";

s1.equals(s2); // true
s1.equals(s3); // false

String s1 = new String("sal");

* String Buffer & String Builder

- ① String Buffer & String Builder both classes are mutable
- ② mutable means changeable
- ③ Ex. String Buffer s1 = new StringBuffer("JavaBy");
s1.append("kiran");

String Builder s2 = new StringBuffer("JavaBy");
s2.append("kiran");

ans
JavaBykiran

Exception &

Exception Handling.

Abnormal Termination

An exception is an unwanted or unexpected event which occurs during the execution of a program i.e. at runtime, that disrupts the normal flow of the program.

Object (parent class of all classes)

Throwable (parent class of exception & error)

↓
(def.) An unwanted unexpected event which occurs at runtime time that disturbs the normal flow of program error

- ① Exceptions are occurred by our programs.
- ② exception are recoverable/we can handle.
- ③ Two types of exception.
 - ① Compile time exception (checked exception)
 - ② Runtime exception (unchecked exception)
- ① because of lack of system resources.
- ② Not recoverable error
- ③ Runtime exception (unchecked exception)

exception (Include classes)

Catch (Exception e)

ClassNotFoundException

NoSuchMethodException

IOException

SQLException

RemoteException

FileNotFoundException

IOException

InterruptedException

RuntimeException

ArithmeticException

ClassCastException

NullPointerException

IndexOutOfBoundsException

ArrayIndexOutOfBoundsException

StringIndexOutOfBoundsException

Compile time

exception

(checked exception)

RuntimeException (unchecked)

Note → exception always occur at runtime.

Error
Virtual machine Error
↳ Stack overflow Error
↳ Out of memory Error

Linkage Error
↳ Version Error
↳ Class format Error

Assertion Error.

(checked)

Compile time exception

(Difference)

(unchecked)

Runtime exception

→ Those exceptions which are checked by compiler at the time of compile time is known as compile time exceptions.

→ The program gives a compilation error if a method throws a checked exception.

→ If some code within a method throws a checked exception, then the method must either handle the exception or it must specify the exception using throws keyword.

⇒ A checked exception occurs when the chances of failure are too high.

→ They are direct subclass of exception class but do not inherit from RuntimeException.

→ Unchecked exceptions are the exceptions that are not checked at compile time.

→ The program compiles fine because the compiler is not able to check the exception.

→ A method is not forced by compiler to declare the unchecked exceptions thrown by its implementation. Generally, such methods almost always do not declare them, as well.

→ Unchecked exception occurs mostly due to programming mistakes.

→ They are direct subclass of RuntimeException class.

* Exception Handling:-

ex. class Test {
 p.s.v.m.g

```
int a = 100, b = 0;  
int c = a/b;  
syso(c);  
}
```

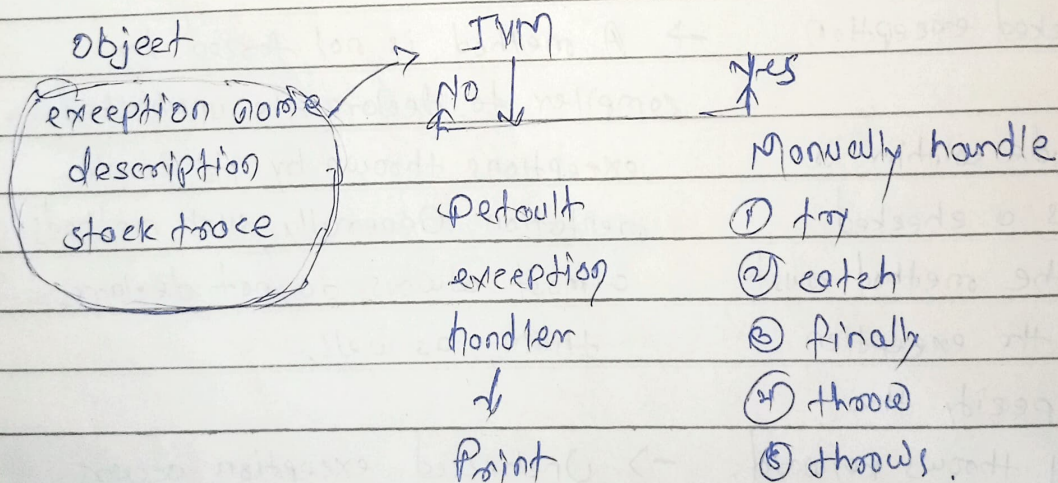
}

→ whenever there is exception, the method in which exception occurs will create an object & that object will

store → ① exception name

② description

③ stack trace (line)



→ Defining the alternative way to continue rest of the my programme so that my programme terminates normally

→ defining the alternative way to recover the exception.
Normal termination.

* try & catch syntax:

```
try  
{  
    // risky code  
}
```

```
} catch (ExceptionClassName ref. var. name)
```

```
{  
    // handling code  
}
```

(if there is exception in try in try block catch block will execute)

ex. class Test {
 p.s. v.m. {
 try

```
{  
    int a = 100, b = 0, z;  
    z = a/b;  
    syso();  
}
```

```
} catch (ArithmeticException e)
```

```
{  
    syso(e); // syso("you cannot divide by zero");  
}
```

```
}  
}
```

→ O/P = java.lang.ArithmeticException: / by zero

← you can not divide by zero